

Project 01: The Searchin' Pac-Man

Search implementation, measurement, and critical assessment

Team member	Stony Brook ID
Param Shah	117369485
Madhura Chitale	117420795

Date: September 13, 2026

1. Scope and implementation

We implemented Questions 1-7 in `search.py` and `searchAgents.py`, covering depth-first search (DFS), breadth-first search (BFS), uniform-cost search (UCS), A* search, the CornersProblem state space, a corners heuristic, and a food heuristic. The project framework, public interfaces, expansion counters, and Berkeley attribution were retained. The assignment requires a report containing nodes expanded, memory usage, runtime, and a critical comparison of the search methods; the following sections summarize those measurements and observations.

Algorithms and state representations

DFS uses `util.Stack` and marks a state when it is removed for expansion. **BFS** uses `util.Queue` and marks states when they are enqueued. **UCS** is implemented as A* with a zero heuristic. **A*** uses `util.PriorityQueue` with priority $g+h$, tracks the best known path cost to each state, ignores stale queue entries, and reopens a state if a cheaper route is discovered.

A `CornersProblem` state contains only Pac-Man's position and an immutable tuple indicating which of the four corners have been visited. The corners heuristic computes the minimum Manhattan tour through the remaining corners, which is the exact solution of the wall-free relaxation. The food heuristic is **distance to the nearest remaining food + a minimum spanning tree (MST) over remaining food** using exact maze distances. Maze-distance maps and MST costs are cached per problem.

Measurement methodology

Measurements in the supplied draft were collected on **CPython 3.10.10** under Windows 10.0.26200. The assignment recommends Python 3.11; the reported values therefore characterize the measured environment rather than a universal runtime baseline. Search times are medians of three fresh searches measured with `time.perf_counter`; heuristic caches are cold at the start of each run and garbage collection is performed before measurement. Timing excludes imports, layout loading, graphics, and plan execution.

Peak memory is measured in a separate fresh run with `tracemalloc` and includes search allocations, bookkeeping, and heuristic caches, while pre-existing interpreter and maze memory are excluded. One KiB is 1024 bytes. Node counts are read from `problem._expanded` before plan replay. Auxiliary BFS work performed inside heuristics contributes to runtime and memory, but not to the problem expansion counter.

Validation summary: all 28 benchmark cases in the measurement suite completed legal-action replay and reached their goal. Additional targeted tests covered start-at-goal, unreachable goals, cycles, cheaper duplicate paths, A* state reopening, corner-state independence, and heuristic consistency.

2. Fixed-position search results

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyMaze	DFS	10	15	0.074	3.9
tinyMaze	BFS	8	15	0.071	3.9
tinyMaze	UCS	8	15	0.077	4.2
tinyMaze	A* Manhattan	8	14	0.083	4.2
mediumMaze	DFS	130	146	0.581	30.9
mediumMaze	BFS	68	269	0.974	39.0
mediumMaze	UCS	68	269	1.106	41.1
mediumMaze	A* Manhattan	68	221	0.997	38.5
bigMaze	DFS	210	390	1.581	116.8
bigMaze	BFS	210	620	2.452	103.3
bigMaze	UCS	210	620	2.657	90.1
bigMaze	A* Manhattan	210	549	2.714	88.8
openMaze	DFS	298	576	3.229	562.5
openMaze	BFS	54	682	2.642	99.5
openMaze	UCS	54	682	3.106	123.1
openMaze	A* Manhattan	54	535	2.466	85.6

Solution quality

On all four unit-cost fixed-position mazes, BFS, UCS, and A* agree on the shortest path length. DFS does not guarantee shortest paths: it takes 130 steps on mediumMaze compared with 68 for BFS/UCS/A*, and 298 steps on openMaze compared with 54. The mediumMaze DFS length differs from the assignment's approximate examples because DFS is sensitive to successor ordering; the implementation preserves the supplied order and returns a legal graph-search solution.

Search effort and heuristic value

A* Manhattan reduces the number of expanded nodes relative to UCS from 620 to 549 on bigMaze (about **11.5%**) and from 682 to 535 on openMaze (about **21.6%**). The wall-clock effect is smaller because heuristic evaluation and priority-queue bookkeeping add overhead: on bigMaze, A* expands fewer nodes but is slightly slower than UCS in this microbenchmark (2.714 ms vs. 2.657 ms). On openMaze, the lower expansion count is large enough that A* is both smaller and faster (2.466 ms vs. 3.106 ms).

Memory and interpretation

DFS stores full action lists in fringe entries, so it is not automatically the most memory-efficient method: on openMaze it reaches 562.5 KiB, much higher than BFS (99.5 KiB), UCS (123.1 KiB), or A* (85.6 KiB). Expanded states describe planning work rather than physical movement; Pac-Man executes only the returned plan and does not traverse every explored square.

Takeaway

For unit-cost shortest-path problems, BFS is the simplest reliable baseline. UCS matches shortest-path behavior on unit costs but is designed for nonuniform costs. A* can reduce unnecessary exploration when its heuristic is informative, but lower node count does not automatically imply lower runtime because heuristic computation has a cost.

3. Corners, food, and weighted-cost search

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyCorners	BFS	28	252	1.419	85.1
tinyCorners	A* corners	28	159	3.009	84.3
mediumCorners	BFS	106	1966	11.189	678.3
mediumCorners	A* corners	106	741	19.303	317.9
testSearch	UCS	7	14	0.419	16.8
testSearch	A* food MST	7	7	0.285	13.1
tinySearch	UCS	27	5057	219.103	8400.2
tinySearch	A* food MST	27	89	5.945	264.4
trickySearch	UCS	60	16688	1485.511	48296.3
trickySearch	A* food MST	60	255	30.424	1065.6
mediumDottedMaze	UCS east	74	186	0.987	41.1
mediumScaryMaze	UCS west	92	108	0.609	30.1

CornersProblem and corners heuristic

BFS and A* both find a 28-step solution on tinyCorners and a 106-step solution on mediumCorners, supporting optimality of the A* result. On mediumCorners, the heuristic reduces expansions from 1,966 to 741 (**62.3%**) and peak measured memory from 678.3 KiB to 317.9 KiB (**53.1%**). Runtime increases from 11.189 ms to 19.303 ms because each heuristic evaluation enumerates the small set of possible remaining-corner orders. The assignment awards full Q6 heuristic credit below 1,200 expansions; 741 is also below the 800-node "doing great" marker.

FoodSearchProblem and MST heuristic

The food heuristic preserves the optimal costs produced by UCS: 7 on testSearch, 27 on tinySearch, and 60 on trickySearch. Its effect on difficult food search is substantial. On tinySearch, expansions fall from 5,057 to 89 (about **98.2%**). On trickySearch, expansions fall from 16,688 to 255 (**98.5%**), runtime from 1,485.511 ms to 30.424 ms (**98.0%**), and peak memory from 48,296.3 KiB to 1,065.6 KiB (**97.8%**). The assignment's Q7 extra-credit threshold is fewer than 7,000 expansions, so the measured 255-node result is comfortably below that threshold, subject to the heuristic remaining admissible and consistent.

Weighted costs

The table reports path length in steps, not weighted path cost. StayEastSearchAgent uses UCS to exploit a cost function that rewards eastern positions, taking 74 steps with weighted cost approximately **1.0009765838040044**. StayWestSearchAgent takes 92 steps with weighted cost **68,719,479,864**. These dramatically different path costs confirm why BFS is not sufficient when step costs vary and why UCS is the appropriate uninformed algorithm for the weighted layouts.

4. Correctness, heuristic justification, and critical assessment

Correctness checks

The measurement suite reports that all 28 benchmark searches returned legal action sequences and reached the intended goal. Targeted checks also covered a start state that is already a goal, unreachable goals, cyclic graphs, duplicate states reached with cheaper costs, and A* reopening under an admissible but inconsistent heuristic. BFS, UCS, and A* also solved the supplied eight-puzzle instance in the validation setup.

Heuristic checks and why the bounds are valid

For the corners heuristic, removing walls relaxes the original problem. The shortest Manhattan tour through the remaining corners can therefore never exceed the true maze completion cost. Because a legal unit move in the original maze is also a legal unit move in the wall-free relaxation, the relaxed optimal cost changes by at most one per step, giving a consistent lower bound. The heuristic is nonnegative and returns zero when no corners remain.

For the food heuristic, every complete solution must first connect Pac-Man to the remaining food set and must ultimately connect all remaining food locations. The exact maze distance from the current position to the nearest food plus the MST weight of the remaining foods is therefore a lower bound. With the food set unchanged, nearest-food distance can fall by at most one after a legal move. When a food dot is eaten, that removed dot can be attached to an MST of the remaining dots by its nearest maze-distance edge, establishing the same one-step consistency condition. The bound is nonnegative and vanishes at the goal.

The supplied validation notes report exhaustive consistency checks over **912 transitions across 384 reachable tinyCorners states** and **17,732 transitions across 8,100 tinySearch states**. All checked heuristic values were nonnegative and zero at goals, and A* and UCS agreed on solution lengths. Separate checks verified that a starting corner is counted as visited and that successor generation does not mutate parent states.

Limitations

No official autograder or test_cases directory was included with the measured package, so the report does not claim an official autograder score. Python 3.11, which the assignment recommends, was not available in the original measurement environment; runtime and memory values are therefore environment-specific. The measurements are most useful for relative comparisons among methods under the same setup rather than as absolute performance claims across machines.

Critical assessment

DFS is simple and can reach a goal with relatively few expansions, but its returned path can be much longer than optimal and its memory usage can still grow when complete action prefixes are stored. **BFS** is a strong baseline for unit-cost shortest paths, but it can keep a large frontier. **UCS** generalizes shortest-path search to nonnegative weighted costs and is necessary for the StayEast/StayWest layouts. **A*** combines UCS-style cost awareness with problem knowledge; its usefulness depends on whether the saved expansions outweigh heuristic computation.

The experiments illustrate that tradeoff clearly. Manhattan A* provides moderate node savings on fixed-position mazes. The corners heuristic saves many expansions and memory but is slower than BFS because heuristic evaluation is comparatively expensive. The food MST heuristic is where A* is most effective: its stronger lower bound makes a difficult combinatorial problem dramatically smaller in nodes, time, and memory. Consequently, search strategies should be compared across **solution quality, node expansions, runtime, and memory together**, rather than ranked by any single statistic.

Conclusion. The implementations satisfy the intended roles of DFS, BFS, UCS, and A*. The abstract CornersProblem state avoids irrelevant game information, while both heuristics preserve optimal solution lengths in the measured tests. The strongest empirical result is the food MST heuristic, which reduces trickySearch from 16,688 to 255 expanded nodes while retaining the 60-step optimal solution.